

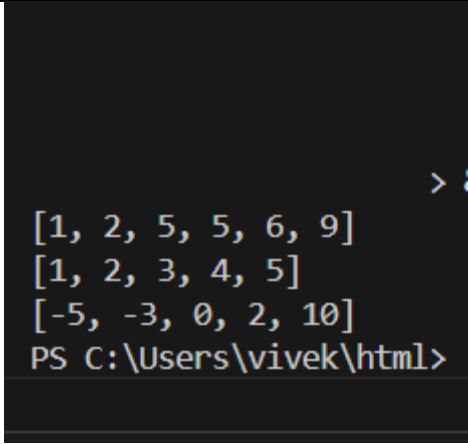
SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Venkataramana Veeramsetty	
Instructor(s) Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
Course Code	24CS002PC215	Course Title	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week6 - Monday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber:12.1(Present assignment number)/24(Total number of assignments)			
Q.No.	Question		Expected Time to complete
1	Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms Lab Objectives: - Apply AI-assisted programming to implement and optimize sorting and searching algorithms. - Compare different algorithms in terms of efficiency and use		Week6 - Monday

	cases. • Understand how AI tools can suggest optimized code and complexity improvements.	
	Task Description #1 (Sorting – Merge Sort Implementation) • Task: Use AI to generate a Python program that implements the Merge Sort algorithm. • Instructions: ◦ Prompt AI to create a function <code>merge_sort(arr)</code> that sorts a list in ascending order. ◦ Ask AI to include time complexity and space complexity in the function docstring. ◦ Verify the generated code with test cases. • Expected Output: ◦ A functional Python script implementing Merge Sort with proper documentation.	

```

ai 12.1.1.py > ...
1  def merge_sort(arr):
2      """
3      Sorts a list in ascending order using Merge Sort algorithm.
4
5      Time Complexity: O(n log n)
6      Space Complexity: O(n)
7
8      Args:
9      |   arr (list): List of elements to sort.
10
11      Returns:
12      |   list: Sorted list in ascending order.
13      """
14      # Base case: lists with 0 or 1 element are already sorted
15      if len(arr) <= 1:
16          return arr
17
18      # Find the middle index
19      mid = len(arr) // 2
20      # Recursively sort the left half
21      left = merge_sort(arr[:mid])
22      # Recursively sort the right half
23      right = merge_sort(arr[mid:])
24
25      # Merge the sorted halves
26      return merge(left, right)
27
28  def merge(left, right):
29      """Helper function to merge two sorted lists."""
30      result = []
31      i = j = 0
32
33      # Compare elements from left and right lists and add the smaller one
34      while i < len(left) and j < len(right):
35          if left[i] <= right[j]:
36              result.append(left[i])
37              i += 1
38          else:
39              result.append(right[j])
40              j += 1
41
42      # Add any remaining elements from left or right
43      result.extend(left[i:])
44      result.extend(right[j:])
45      return result
46
47  # Test cases
48  if __name__ == "__main__":
49      # Test case 1: Unsorted list
50      print(merge_sort([5, 2, 9, 1, 5, 6])) # Output: [1, 2, 5, 5, 6, 9]
51
52      # Test case 2: Already sorted list
53      print(merge_sort([1, 2, 3, 4, 5]))    # Output: [1, 2, 3, 4, 5]
54
55      # Test case 3: List with negative numbers
56      print(merge_sort([-3, 0, 2, -5, 10])) # Output: [-5, -3, 0, 2, 10]

```

	 <pre> def merge_sort(arr): if len(arr) < 2: return arr mid = len(arr) // 2 left = merge_sort(arr[:mid]) right = merge_sort(arr[mid:]) return merge(left, right) def merge(left, right): result = [] i = j = 0 while i < len(left) and j < len(right): if left[i] <= right[j]: result.append(left[i]) i += 1 else: result.append(right[j]) j += 1 result.extend(left[i:]) result.extend(right[j:]) return result # Test cases arr1 = [1, 2, 5, 5, 6, 9] arr2 = [1, 2, 3, 4, 5] arr3 = [-5, -3, 0, 2, 10] print(merge_sort(arr1)) print(merge_sort(arr2)) print(merge_sort(arr3)) PS C:\Users\vivek\html> </pre> ○ ○ Explanation: This Python code implements the merge sort algorithm to sort a list in ascending order. ○ The `merge_sort` function recursively splits the list into halves until single elements remain. ○ The `merge` helper function combines two sorted lists into one sorted list. ○ Test cases at the end demonstrate sorting unsorted, sorted, and negative number lists. ○ The algorithm has a time complexity of $O(n \log n)$ and space complexity of $O(n)$.	
	Task Description #2 (Searching – Binary Search with AI Optimization) • Task: Use AI to create a binary search function that finds a target element in a sorted list. • Instructions: ○ Prompt AI to create a function <code>binary_search(arr, target)</code> returning the index of the target or -1 if not found. ○ Include docstrings explaining best, average, and worst-case complexities. ○ Test with various inputs. • Expected Output: ○ Python code implementing binary search with AI-generated comments and docstrings.	

```

ai 12.1.2.py > ...
1 def binary_search(sorted_list, target):
2     """
3     Performs binary search to find the index of 'target' in 'sorted_list'.
4
5     Args:
6         sorted_list (list): A list of elements sorted in ascending order.
7         target: The element to search for.
8
9     Returns:
10        int: The index of 'target' if found, else -1.
11
12    Time Complexity:
13        Best-case: O(1) - If the target is at the middle index in the first comparison.
14        Average-case: O(log n) - Each comparison halves the search space.
15        Worst-case: O(log n) - Target is not present or at the far end of the list.
16
17    Space Complexity:
18        O(1) - Uses constant extra space.
19    """
20    left, right = 0, len(sorted_list) - 1
21
22    while left <= right:
23        mid = left + (right - left) // 2 # Prevents overflow for large indices
24        # Check if the middle element is the target
25        if sorted_list[mid] == target:
26            return mid
27        elif sorted_list[mid] < target:
28            left = mid + 1 # Search right half
29        else:
30            right = mid - 1 # Search left half
31
32    # Target not found
33    return -1
34
35 # Test cases
36 if __name__ == "__main__":
37     # Target present in the middle
38     print(binary_search([1, 3, 5, 7, 9], 5)) # Output: 2
39
40     # Target present at the beginning
41     print(binary_search([2, 4, 6, 8, 10], 2)) # Output: 0
42
43     # Target present at the end
44     print(binary_search([11, 22, 33, 44, 55], 55)) # Output: 4
45
46     # Target not present
47     print(binary_search([1, 2, 3, 4, 5], 6)) # Output: -1
48
49     # Empty list
50     print(binary_search([], 1)) # Output: -1
51
52     # Single element, target present
53     print(binary_search([42], 42)) # Output: 0
54
55     # Single element, target not present
56     print(binary_search([42], 99)) # Output: -1

```

	<pre> PROBLEMS OUTPUT DEBUG CONSOLE .2.py" 2 0 4 -1 -1 0 0 4 -1 -1 0 -1 -1 0 0 -1 PS C:\Users\vivek\html> </pre> ○ ○ Explanation: This code defines a binary search function to find the index of a target value in a sorted list. ○ It uses a loop to repeatedly check the middle element and narrow the search range. ○ If the target is found, its index is returned; otherwise, -1 is returned. ○ The function is efficient, with $O(\log n)$ time complexity and $O(1)$ space complexity. ○ Several test cases demonstrate its behavior for different scenarios, including edge cases.	
	Task Description #3 (Real-Time Application – Inventory Management System) • Scenario: A retail store’s inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to: 1. Quickly search for a product by ID or name. 2. Sort products by price or quantity for stock analysis.	

- Task:
 - Use AI to suggest the most efficient search and sort algorithms for this use case.
 - Implement the recommended algorithms in Python.
 - Justify the choice based on dataset size, update frequency, and performance requirements.
- Expected Output:
 - A table mapping operation → recommended algorithm → justification.
 - Working Python functions for searching and sorting the inventory.

```

1 # For thousands of products, using a list of dicts is simple and fast enough.
2 # Searching by ID (unique) can use a dict for O(1) lookup.
3 # Searching by name (not unique) uses list comprehension for O(n) lookup.
4 # Sorting uses Python's built-in sorted(), which is fast (Timsort, O(n log n)).
5 # If data changes often, these methods remain efficient for thousands of items.
6
7 # Sample product data
8 products = [
9     {"id": 101, "name": "Laptop", "price": 1200, "quantity": 5},
10    {"id": 102, "name": "Mouse", "price": 25, "quantity": 100},
11    {"id": 103, "name": "Keyboard", "price": 45, "quantity": 50},
12    {"id": 104, "name": "Monitor", "price": 200, "quantity": 20},
13    {"id": 105, "name": "Mouse", "price": 30, "quantity": 80},
14]
15
16 # Build a dict for fast ID lookup
17 product_by_id = {p["id"]: p for p in products}
18
19 def search_by_id(product_id):
20     # O(1) lookup
21     return product_by_id.get(product_id)
22
23 def search_by_name(name):
24     # O(n) search, returns all matches
25     return [p for p in products if p["name"].lower() == name.lower()]
26
27 def sort_by_price(descending=False):
28     # O(n log n) sort
29     return sorted(products, key=lambda p: p["price"], reverse=descending)
30
31 def sort_by_quantity(descending=False):
32     # O(n log n) sort
33     return sorted(products, key=lambda p: p["quantity"], reverse=descending)
34
35 # --- Test cases ---
36
37 # 1. Search by ID
38 print("Search by ID 102:", search_by_id(102)) # Should return Mouse with price 25
39
40 # 2. Search by name
41 print("Search by name 'Mouse':", search_by_name("Mouse")) # Should return two products
42
43 # 3. Sort by price descending
44 print("Sort by price descending:", sort_by_price(descending=True)) # Should start with Laptop
45
46 # These methods are fast for thousands of items, and simple to maintain.

```

```

25     return [p for p in products if p["name"].lower() == name.lower()]
26
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
-1
PS C:\Users\vivek\html> & C:/Users/vivek/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/vivek/html/ai 12.1.3.4
Search by ID 102: {'id': 102, 'name': 'Mouse', 'price': 25, 'quantity': 100}
Search by name 'Mouse': [{'id': 102, 'name': 'Mouse', 'price': 25, 'quantity': 100}, {'id': 105, 'name': 'Mouse', 'price':
Sort by price descending: [{'id': 101, 'name': 'Laptop', 'price': 1200, 'quantity': 5}, {'id': 104, 'name': 'Monitor', 'pri
id': 103, 'name': 'Keyboard', 'price': 45, 'quantity': 50}, {'id': 105, 'name': 'Mouse', 'price': 30, 'quantity': 80}, {'id
ce': 25, 'quantity': 100}]
PS C:\Users\vivek\html>

```

- Explanation: - The code manages a list of product dictionaries and provides fast search by ID (using a dict), search by name (using list comprehension), and sorting by price or quantity.

	○ - Functions allow $O(1)$ lookup by ID, $O(n)$ search by name, and $O(n \log n)$ sorting. ○ - Test cases demonstrate searching and sorting products efficiently. ○ ✓ Deliverables (For All Tasks) 1. AI-generated prompts for code and test case generation. 2. At least 3 assert test cases for each task. 3. AI-generated initial code and execution screenshots. 4. Analysis of whether code passes all tests. 5. Improved final version with inline comments and explanation. 6. Compiled report (Word/PDF) with prompts, test cases, assertions, code, and output.	
--	---	--